Reliable apps crash.



Dusan Stanojevic

tracewayapp.com/blog

A talk about kitchens, warm fridges, and knowing when to close.

Reliable means the same result, every time

A good restaurant serves the same dish to every table. A reliable app gives every request the same correct answer.



A RELIABLE RESTAURANT

Same dish, same quality, order after order. Not just open.

A RELIABLE APP

Same correct answer, request after request. Not just up.

Uptime is necessary. Correctness is the point.

State is what outlives a request

Stateless code starts every request from scratch. Stateful code shares memory between requests.



LIVES FOR ONE REQUEST

- request parameters
- local variables
- the response being built

A failure throws all of it away.

In a kitchen: the order ticket.

SHARED BETWEEN REQUESTS

- in-memory caches and maps
- counters and rate limiters
- singletons and static fields
- connection pools, in-memory sessions

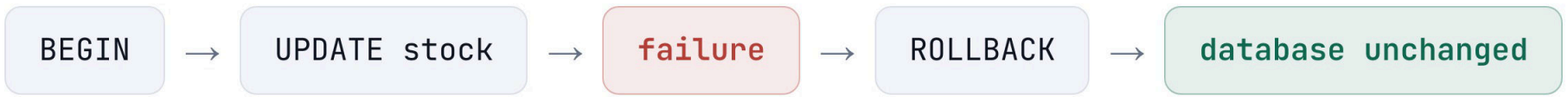
A failure can leave it half updated.

In a kitchen: the fridge and the stock sheet.

Why most apps never hit this

When all shared state lives in a transactional database, a failed request rolls back and leaves nothing behind.

STATE IN A TRANSACTIONAL DATABASE



STATE IN PROCESS MEMORY



This talk is about the second row: caches, counters and maps in memory, plus effects outside the transaction like emails, payments and calls to other services.

Up is not the same as correct

The framework caught the failure. The process is up. The health check is green.



WHAT YOUR DASHBOARD SAYS

Process: running

Health check: 200 OK

Process: still serving

WHAT IS ACTUALLY TRUE

Later requests may reuse state whose invariants no longer hold.

Stopping is safer than serving bad state

A restart is safe only with a trustworthy rebuild. Bad recovery keeps suspect state in service.



CRASH + SUPERVISOR

Closed, but safe

Suspect memory is discarded

Rebuilds to a known-good state

Loud: you know it happened



POORLY RECOVERED BOUNDARY

Open, but wrong

Corrupt state stays in memory

Later requests may inherit it

Quiet: customers find it first

What we'll cover

Five courses, about thirty minutes, one rule to take home.

— MENU —

Amuse-bouche	Reliability, state, and why closing can win	6 min
I · Invariants	Invariants, side effects, the request boundary	5 min
II · The bug	An in-memory cache that lies	4 min
III · Two fixes	Store less state, or crash and restart	3 min
IV · Boundaries	Choosing where to recover	4 min
V · Supervisors	Restarts, backoff, readiness	3 min
Last call	The rule, and leaving a note	3 min

COURSE 01 / 05

Invariants and side effects.

Before we talk about crashing, we need words for what a crash can break.

TABLE 01 5 MIN

INVARIANTS

- [ ] invariants
- [ ] temporary breaks
- [ ] side effects
- [ ] the request boundary

FIRE ▸

Invariants: rules your state must always follow

“A set of rules that must hold true when it is observed.”

BANK

An account balance never goes below zero.

```
balance >= 0
```

KITCHEN

The walk-in holds what the inventory sheet says.

```
sheet == fridge
```

OUR CACHE

The key counter matches the keys stored.

```
numKeys == len(values)
```



Invariants can break during an update

That is fine while nobody can see it. Restore them before the state is visible again.





DURING THE UPDATE

The counter is updated; the map is not. A lock hides this from other requests.

BEFORE RELEASE

Both are updated before anyone else can look.

A FAILURE

can expose the half-done update when defer or finally releases the lock.

Side effects: everything besides the return value

Unwinding the stack throws away local values. Everything else needs an owner.

EFFECT	GO / JAVA	AFTER A FAILURE, WHO RESTORES IT?	
Local values	`buf := … / var buf = …`	Nobody needs to	They disappear with the stack.
Lock	`mu.Lock() / synchronized`	defer / finally	Released, which also exposes the state.
In-memory write	`numKeys++ / map.put(k, v)`	Nobody	Unless your code restores it.
Database write	`INSERT inside a transaction`	The database	On rollback.
Outside the process	`HTTP call, file, email`	Nobody	It already left. Reconcile it.

Recovery must leave valid state: discard, roll back, or reconcile affected work.

Frameworks recover at the request boundary

A failed handler becomes a 500, and the server keeps serving. Shared state stays as the failure left it.

Framework catches here: log, 500, next request

YOUR HANDLER

`handle(request)` — panic / exception

writes →

SHARED STATE

`counters, maps, singletons`

outside the boundary, not rolled back

WHAT HAPPENS WHEN A HANDLER FAILS

01 **Request** — arrives at the handler

→

02 **Exception** — panic or throw mid-handler

→

03 **Unwind** — defer / finally run, locks released

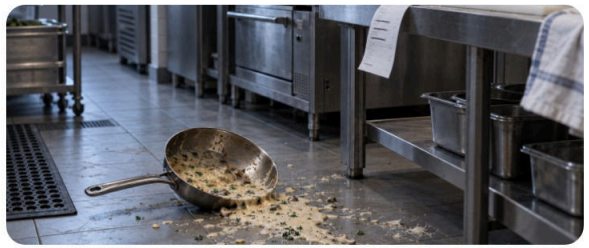
→

04 **Recover** — framework logs it, returns 500

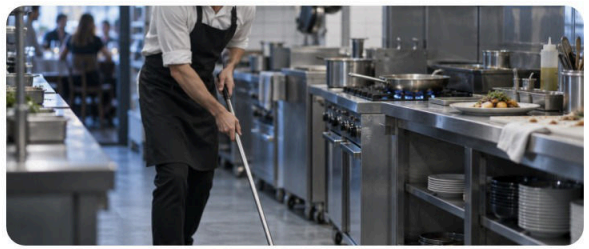
→

05 **Next request** — same process, same memory

No step checks the shared state.



`panic` / `throw` drops the pan.



`recover` / `catch` mops the floor.

Nobody checks what was left on the stove.

COURSE 02 / 05

A cache that lies.

A tiny in-memory cache, a simulated failure, and a server that keeps smiling.

TABLE 02 4 MIN

THE BUG

- [ ] Cache.Set
- [ ] curl ×3
- [ ] 2 ≠ 1

FIRE ▸

Two fields that must agree

A small in-memory cache behind an HTTP handler. numKeys and the map both describe how many keys exist.

```
type Cache struct {
    mu      sync.RWMutex
    values  map[string]string
    numKeys int                          A
}

func (c *Cache) Set(key, value string) {
    c.mu.Lock()
    defer c.mu.Unlock()

    if _, exists := c.values[key]; !exists {
        c.numKeys++                      1
    }
    if value == "PANIC" {
        panic("simulated failure")       2
    }
    c.values[key] = value                3
}
```

A Duplicated state

numKeys repeats what len(values) already knows.

1 First write

The counter is incremented.

2 Failure

A panic between the two writes.

3 Never runs

The map is not updated.

Java has the same bug: a `synchronized` method that increments, then throws before `put`.

The server is up. The cache is wrong.

Three requests. One recovered panic. Wrong state until repair or restart.

```
$ curl -X PUT localhost:8080/cache/first  -d 'hello'
$ curl -X PUT localhost:8080/cache/second -d 'PANIC'
$ curl localhost:8080/stats
```

SERVER LOG

```
http: panic serving 127.0.0.1:52144: simulated failure
goroutine 34 [running]: …
```

GET /STATS

```
reported_keys=2 stored_keys=1
```

REPORTED VS STORED

2 ≠ 1

Every later request sees this until the process restarts.

The health check still says 200 OK.

The bad state outlives the request



"When the fridge breaks and has been warm all night, the kitchen does not keep serving just because the doors are open. It closes, throws out everything it cannot trust, restocks, and only then seats guests again."

Process	`up`
Health check	`green`
Invariants	`broken`

COURSE 03 / 05

Two fixes.

Remove the state that can break, or stop the process when you can't trust it.

TABLE 03 3 MIN

TWO FIXES

- [ ] store less state
- [ ] crash and restart

FIRE ▸

One fix: stop storing what you can derive

One possible fix for this cache: remove the duplicated state so nothing can fall out of sync.

```
type Cache struct {
    mu     sync.RWMutex
    values map[string]string
}

func (c *Cache) Set(key, value string) {
    c.mu.Lock()
    defer c.mu.Unlock()

    if value == "PANIC" {
        panic("simulated failure")
    }
    c.values[key] = value
}

func (c *Cache) Len() int {
    c.mu.RLock()
    defer c.mu.RUnlock()
    return len(c.values)
}
```

OPTION 1 OF 2

WHY IT WORKS HERE

The count is derived from the map. The panic happens before the only shared write, so recovery at the request is safe for this path.

LIMITS

Works only when state can be derived or removed. Every new shared write means checking again.

No safe recovery? Crash.

The best default for stateful apps: record the failure, exit, and restart from a clean state.

GO

```go
func setOrExit(cache *Cache, key, value string) {
    defer func() {
        if failure := recover(); failure != nil {
            log.Printf("fatal cache panic: %v\n%s",
                failure, debug.Stack())
            os.Exit(1)
        }
    }()
    cache.Set(key, value)
}
```

JAVA

```java
void setOrExit(Cache cache, String key, String value) {
    try {
        cache.set(key, value);
    } catch (Throwable failure) {
        log.error("fatal cache failure", failure);
        Runtime.getRuntime().halt(1);
    }
}
```

01

Record it

Log the failure and the stack.

02

Exit

Stop taking requests on suspect state.

03

Restart clean

A supervisor starts a fresh process.

OPTION 2 OF 2

Safe here because the cache is disposable. Exiting does not undo effects that already left.

COURSE 04 / 05

Choosing a boundary.

Recover in the smallest part of the system that owns the whole mess.

TABLE 04 4 MIN

BOUNDARIES

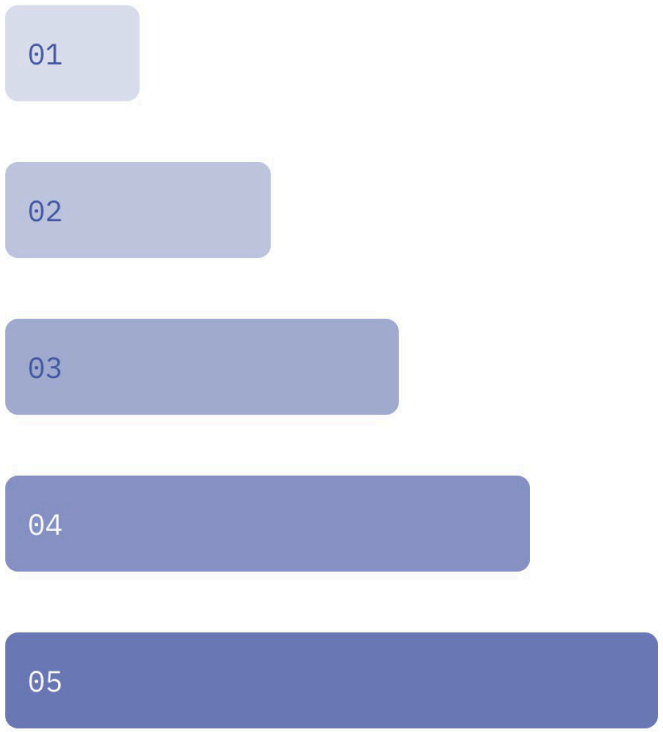
- [ ] operation → process
- [ ] background workers
- [ ] reporting ≠ repair

FIRE ▸

From one operation to the whole process

Recover at the narrowest boundary that can establish valid state.

NARROW → BROAD



BOUNDARY	EXAMPLE	WHEN RECOVERY IS SAFE
Operation	Parsing one value	Only local values touched. Discard the result.
Request	One HTTP request in one transaction	Surviving state is valid or reliably restored.
Durable job	A queue message with retries	Retry safely; reconcile ambiguous outcomes.
Worker	A goroutine or thread with its own state	Discard private state; account for other effects.
Process	The whole service	Discards memory. External effects survive.

Background workers need their own boundary

Request-level recovery does not cover goroutines or threads you start yourself.

GO

```go
go func() {
    defer func() {
        if failure := recover(); failure != nil {
            log.Printf("worker panic: %v\n%s",
                failure, debug.Stack())
            restart <- struct{}{}
        }
    }()
    worker()
}()
```

Without this, a panic in any goroutine **kills the whole process**.

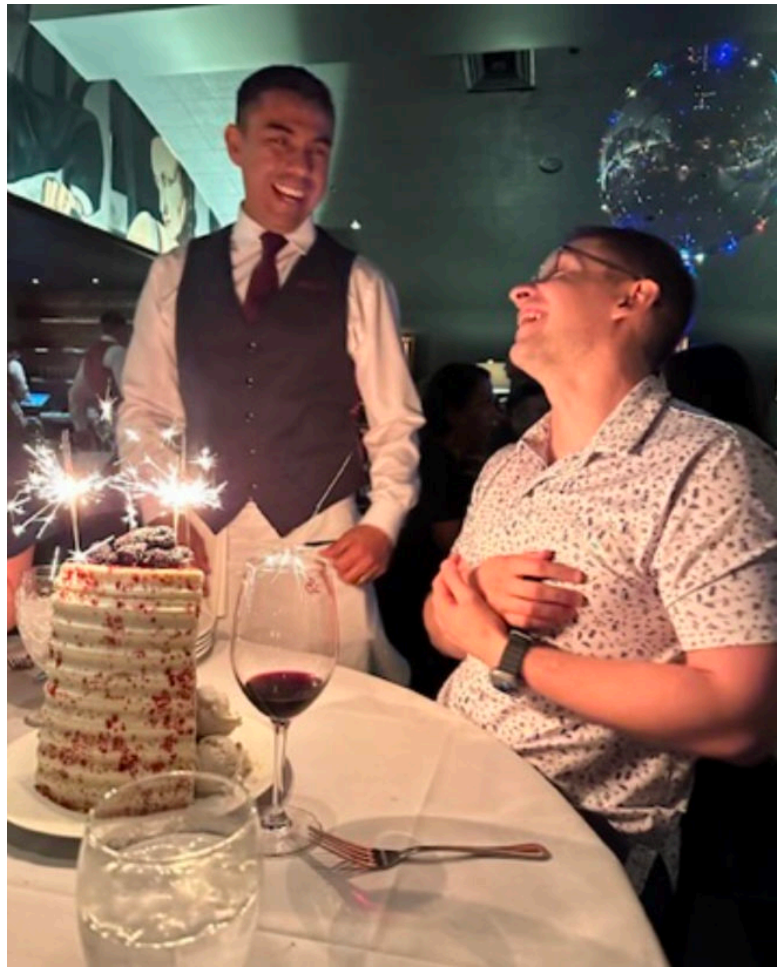
JAVA

```java
executor.execute(() -> {
    try {
        worker();
    } catch (Throwable failure) {
        log.error("worker failed", failure);
        restart.offer(failure);
    }
});
```

Without this, an uncaught exception **ends only that thread**. With `submit()` it waits in a Future that may never be read.

ONLY IF the worker's state is private or safely restored. A new worker on the same broken map is not a fresh start.

Reporting an error does not fix the state



“Reporting is not repair. Continue only when the surviving state is valid.”

IF YOU NEED TO REPORT: REPORT, THEN EXIT

```go
defer func() {
    if r := recover(); r != nil {
        report(r, debug.Stack())
        flushWithDeadline()
        os.Exit(1)
    }
}()
```

```java
catch (Throwable t) {
    report(t);
    flushWithTimeout();
    Runtime.getRuntime().halt(1);
}
```

COURSE 05 / 05

Supervisors.

Closing only works if a supervisor reopens, carefully.

TABLE 05 3 MIN

SUPERVISORS

- [ ] detect
- [ ] restart
- [ ] back off
- [ ] ready check

FIRE ▸

Crashing is only safe with a supervisor

systemd, Docker or Kubernetes decide what happens after the process exits.

1. **Detect**
   Notice the process stopped.
   `Process exit status`

2. **Restart**
   Start fresh, with clean memory.
   `Restart=on-failure`

3. **Back off**
   Delay repeated restarts; use backoff for persistent failure.
   `Fixed delay / restart backoff`

4. **Send traffic when ready**
   No requests until the new process says it is ready.
   `readinessProbe`



`systemd` `docker --restart` `kubelet`

War story: the restart that never came



NO WRAPPER

Background goroutines with no recover — any panic kills the process.

SLOW SUPERVISOR

A supervisor that was misconfigured and slow to restart.

RESULT

One goroutine panic took production down.

Three lines to take home

01 **Recover at the narrowest boundary that can establish valid state.**

Throw out the dish, not the restaurant.

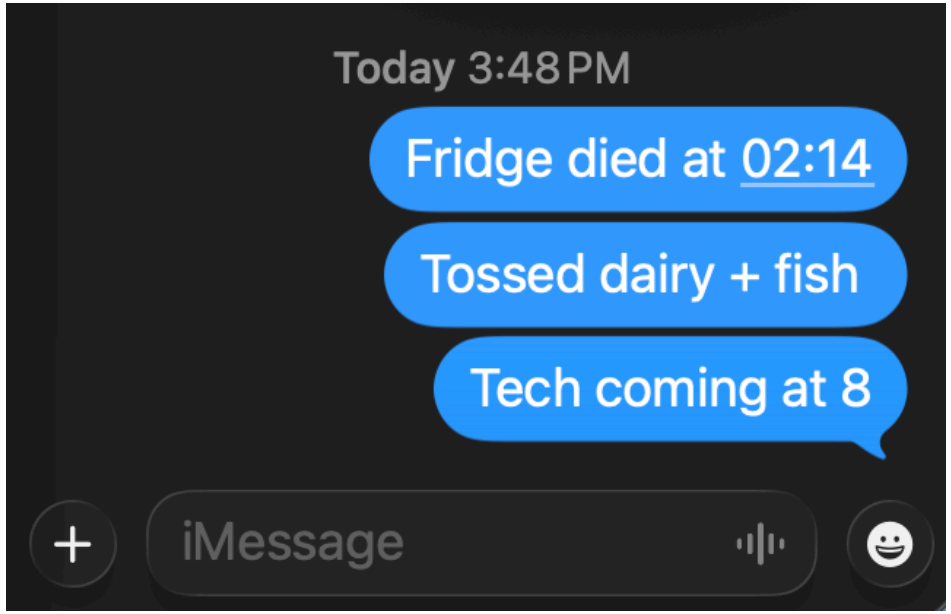
02 **Otherwise, stop the component. Restart only through a trusted recovery path.**

If you can't tell what's spoiled, close.

03 **Then go read the panic — the crash was the easy part.**

Find out why the fridge died.

Reliable apps crash. Good ones leave a note.



The morning shift needs to know what happened at 2am. So does the on-call engineer.

The panic and its stack
what broke, and where

Linked traces and logs
what the request was doing

A page to the right person
who brings it back

That's what we build at **Traceway**.

LAST CALL

Thank you.

Questions? The full write-up, with runnable code, is on the blog.

Dusan Stanojevic

tracewayapp.com/blog

